Livelli di isolamento
Progetto

- Studiare e sperimentare il comportamento di uno o più DBMS (si consiglia Postgres) nella gestione dei livelli di isolamento, con riferimento alle principali anomalie. Si suggerisce di fare riferimento anche ad esempi simili a quelli utilizzati nei lucidi (considerarli tutti, perché potrebbero essere trattati nelle prove d'esame o in quelle intermedie).
- Dal punto di vista realizzativo, si può procedere sviluppando applicazioni in Java con JDBC, oppure attraverso stored procedure oppure anche utilizzando gli ambienti interattivi. Consegnare una breve relazione che illustri i test fatti, mostrando i risultati dei test stessi (file di output o schermate o anche descrizione delle schermate).

Esempio 1 (perdita di aggiornamento)

Start Transaction isolation level XXX	
Read(x)	Start Transaction isolation level XXX
	Read(x)
x=x+100	
Write(x)	
Commit	
	x=x+1
	Write(x)
	Commit

Esempio 1 bis (piccola variante)

Start Transaction isolation level XXX	
Read(x)	
	Start Transaction isolation level XXX
	Read(x)
x=x+100	
Write(x)	
	x=x+1
	Write(x)
Commit	
	Commit

Esempio 2, lettura inconsistente

Start Transaction	
Read(x)	
	Start Transaction
	Read(x)
	x=x+20
	Write(x)
	Commit
Read(x)	
Commit	

Su Postgres

- Usiamo due finestre client sulla stessa base di dati
- Supponiamo che x sia il saldo del conto con `num=1` della relazione conti

```
create schema "20250423livelliIsolamento";
set search_path to "20250423livelliIsolamento";
create table conti (num integer primary key,
                    saldo integer);
insert into conti values (1,0);
```

- Leggiamo con

```
select saldo into app1 from conti where num=1;
```

Read(X)

- E scriviamo con

```
update conti set saldo =
    (select saldo + 100 from app1)
     where num = 1
```

Write(X+100)

- Per ripristinare

```
update conti set saldo = 0 where num = 1
```

Esempio 1-S su Postgres

```
drop table if exists app1;
start transaction
  isolation level serializable;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2;
```

```
start transaction
  isolation level serializable;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
commit
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Esempio 1-RR su Postgres

```
drop table if exists app1;
start transaction
  isolation level
        repeatable read;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2;
start transaction
  isolation level
        repeatable read;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
commit
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Esempio 1-RC su Postgres

```
drop table if exists app1;
start transaction
  isolation level
        read committed;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2;
start transaction
  isolation level
        read committed;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
commit
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Commento

- Nei casi "serializable" e "repeatable read", la seconda transazione fallisce, perché il dato è stato modificato dalla prima, dopo l'inizio della seconda
- Vediamo meglio il comportamento, con una variante che evidenzia due aspetti
  - Lock
  - Verifica dell'ammissibiltà

Esempio 1-S bis su Postgres

```
drop table if exists app1;
start transaction
  isolation level serializable;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2;
start transaction
  isolation level serializable;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
```

```
commit
```

```
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Esempio 1-RC bis su Postgres

```
update conti set saldo = 0 where num = 1;
select * from conti;
drop table if exists app1;
start transaction
  isolation level read committed;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2;
```

```
start transaction
  isolation level read committed;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
```

```
commit
```

```
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Esercizi

- Continuate per conto vostro, con gli altri esercizi nel resto della presentazione, o altri (ad esempio compiti d'esame)
  - Multiversion con Postgres
  - 2PL "a mano"

Vediamo "a mano" qualche esempio 2PL

Esempio 1bis con 2PL

Start Transaction isolation level XXX	
Read(x)	Start Transaction isolation level XXX
	Read(x)
x=x+100 Write(x)	
	x=x+1 Write(x)
Commit	
	Commit

Esempio 1 ter con 2PL

Start Transaction isolation level XXX	
Read(x)	Start Transaction isolation level XXX
x=x+100	
Write(x)	
	Read(x)
Commit	x=x+1
	Write(x)
	Commit

Esempio 1-RC ter su Postgres

```
update conti set saldo = 0 where num
= 1;
select * from conti;
drop table if exists app1;
start transaction
  isolation level read committed;
select saldo into app1
from conti where num=1;
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
drop table if exists app2;
```

```
start transaction
  isolation level read committed;
select saldo into app2
from conti where num=1;
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
```

```
commit
```

```
commit
```

Esempio 1quater con 2PL

```
start transaction
        isolation level serializable
read(x)
x = x + 100;
write(x)
                                        start transaction
                                                isolation level serializable
                                        read(x)

commit                                  x = x +1
                                        write(x)
                                        commit
```

Esempio 1 quater su Postgres (eseguire sul sistema)

```
update conti set saldo = 0;
drop table if exists app1
start transaction
  isolation level serializable;
select saldo into app1
from conti where num=1;
```

```
drop table if exists app2
start transaction
  isolation level serializable;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
select saldo into app2
from conti where num=1;

update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
commit
```

```
commit
```

```
drop table if exists app2
```

```
drop table if exists app1
```

Esempio 1 bis su Postgres

```
start transaction
  isolation level serializable;
select saldo into app1
from conti where num=1;
```

```
start transaction
  isolation level serializable;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
```

```
commit
```

```
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Esempio 1 bis con 2PL

```
start transaction
  isolation level serializable;
select saldo into app1
from conti where num=1;
```

```
start transaction
  isolation level serializable;
select saldo into app2
from conti where num=1;
```

```
update conti set saldo =
  (select saldo + 100 from app1)
     where num = 1;
```

```
update conti set saldo =
  (select saldo + 1 from app2)
     where num = 1;
```

```
commit
```

```
commit
```

```
drop table if exists app1
```

```
drop table if exists app2
```

Confronto

- Esempio 1bis
  - 2PL
    - va in stallo
  - MVCC (Postgres):
    - gestisce la situazione, abortendo una transazione
- Esempio 1quater
  - 2PL (DB2):
    - con i lock, ha governato la situazione e ha portato a compimento entrambe le transazioni
  - MVCC (Postgres):
    - troppo ottimista, ha fatto avviare entrambe le transazioni e non può portarle a termine (almeno non entrambe)

Esercizio d'esame 27/05/2019

- Sintassi non proprio Postgres

`start transaction`
`  isolation level serializable;`
`select * from impiegati;`	

	`start transaction`
	`  isolation level serializable;`
	`select * from impiegati;`

`insert into impiegati`
`  (matricola, cognome, conteggio)`
`  values (101,'Rossi',1);`
`commit`	

	`insert into impiegati`
	`  (matricola, cognome, conteggio)`
	`  values (102,'Bruni',2);`
	`commit`

`start transaction`
`  isolation level serializable;`
`XX = select count(*) from impiegati;`	

	`start transaction`
	`  isolation level serializable;`
	`YY = select count(*) from impiegati;`

`insert into impiegati`
`  (matricola, cognome, conteggio)`
`  values (101,'Rossi', XX+1);`
`commit`	

	`insert into impiegati`
	`  (matricola, cognome, conteggio)`
	`  values (102,'Bruni', YY+1));`
	`commit`

Esercizio d'esame 27/05/2019 con Postgres scenario 1

```
set search_path to "20250423livelliIsolamento";

drop table if exists impiegati;
create table impiegati
   (matricola integer primary key,
    cognome text,
    conteggio integer);
```

- La lasciamo vuota, senza ennuple

Esercizio d'esame 27/05/2019 con Postgres scenario 1

```
start transaction
  isolation level serializable;
select * from impiegati;
```

```
start transaction
  isolation level serializable;
select * from impiegati;
```

```
insert into impiegati
 (matricola, cognome, conteggio)
values (101,'Rossi',1);
commit
```

```
insert into impiegati
(matricola, cognome, conteggio)
values (102,'Bruni',2);
commit
```

Che cosa succede?

- Proviamo con livelli di isolamento diversi
  - serializable:
    - la seconda transazione viene abortita
  - repetable read (svuotiamo la tabella prima di iniziare)
    - entrambe le transazioni vengono accettate

Esercizio d'esame 27/05/2019 con Postgres scenario 2

```
delete from impiegati;
drop table if exists r1;
start transaction
  isolation level serializable;
select count(*) as C into R1
from impiegati;



insert into impiegati
 (matricola, cognome, conteggio)
select 101,'Rossi',R1.C +1 from R1;
commit
```

```
drop table if exists r2;



start transaction
  isolation level serializable;
select count(*) as C into R2
from impiegati;



insert into impiegati
 (matricola, cognome, conteggio)
select 102,'Bruni', R2.C +1 from R2;
commit
```

Esercizio d'esame 27/05/2019 con Postgres scenario 2

```
delete from impiegati;
drop table if exists r1;
start transaction
  isolation level repeatable read;
select count(*) as C into R1
from impiegati;



insert into impiegati
 (matricola, cognome, conteggio)
select 101,'Rossi',R1.C +1 from R1;
commit
```

```
drop table if exists r2;



start transaction
  isolation level repeatable read;
select count(*) as C into R2
from impiegati;



insert into impiegati
 (matricola, cognome, conteggio)
select 102,'Bruni', R2.C +1 from R2;
commit
```

Esercizio d'esame 27/05/2019 con Postgres scenario 2, esecuzione seriale

```
start transaction
  isolation level serializable;
select into R count(*) C
from impiegati;
insert into impiegati
 (matricola, cognome, conteggio)
values (101,'Rossi',1);
commit
```

```
start transaction
  isolation level serializable;
select * from impiegati;
insert into impiegati
(matricola, cognome, conteggio)
values (102,'Bruni',2);
commit
```

	matricola integer	cognome text	conteggio integer
1	101	Rossi	1
2	102	Bruni	2

Esercizio d'esame 27/05/2019 con Postgres scenario 2, esecuzione concorrente

```
start transaction
  isolation level serializable;
select into R count(*) C
from impiegati;
```

```
start transaction
  isolation level serializable;
select * from impiegati;
```

```
insert into impiegati
 (matricola, cognome, conteggio)
values (101,'Rossi',1);
commit
```

```
insert into impiegati
(matricola, cognome, conteggio)
values (102,'Bruni',2);
commit
```

Che cosa succede?

- Proviamo con livelli di isolamento diversi
  - serializable:
    - la seconda transazione viene abortita
  - repetable read
    - entrambe le transazioni vengono accettate
    - con quale risultato?

	matricola integer	cognome text	conteggio integer
1	101	Rossi	1
2	102	Bruni	1

Perché?

- Il controllo di concorrenza "ignora la semantica delle operazioni"
  - sa solo che ci sono letture e scritture,
  - ma non sa se le letture influenzano le scritture, cosa che
    - avviene in uno scenario (che quindi andrebbe evitato)
    - ma non nell'altro (che non crea problemi).